

Nível de Microarquitetura (parte 3)



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

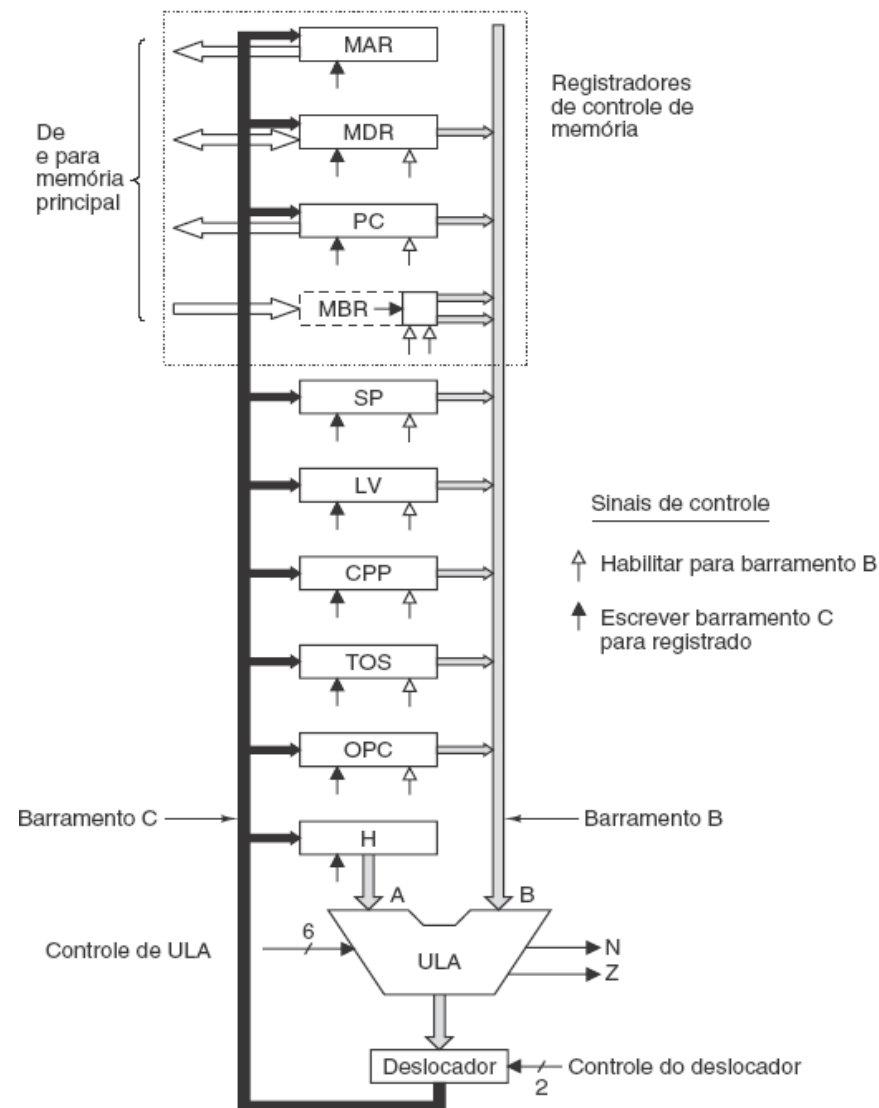
Prof. Carlos Minoru Tamaki

Exemplo de implementação

- Já especificamos a **Microarquitetura** (caminho de dados).
- Já conhecemos a **Macroarquitetura** (IJVM - conjunto de instruções).
- **A questão agora é:** Como um programa interpretador roda no hardware para executar as instruções?

Microinstruções e Notação

- Binário: 36 bits por palavra (difícil de ler e depurar).
- Simbólico: Notação que transmite a **essência** das operações.
- O objetivo é ilustrar conceitos, não necessariamente maximizar flexibilidade.



- Uma abordagem possível é simplesmente fazer uma lista dos sinais que deveriam ser ativados a cada ciclo de relógio. Suponha que em um determinado ciclo queremos incrementar o valor de SP. Também queremos iniciar uma operação de leitura, e queremos que a próxima instrução seja a que reside na localização 122 no armazenamento de controle. Poderíamos escrever:
- ReadRegister = SP; ALU = INC; WSP; Read; NEXT_ADDRESS = 122
- Onde WSP significa 'escreva o registrador SP'.

- Essa notação é completa, mas difícil de entender. Em vez disso, combinaremos as operações de modo natural e intuitivo para captar o efeito daquilo que está acontecendo:
- $SP = SP + 1$; rd

- Vamos dar a nossa Micro Assembly Language de alto nível o nome 'MAL' (palavra francesa que significa 'doente', o que com certeza você ficará se tiver de escrever muito código utilizando essa linguagem). A MAL é talhada para refletir as características da microarquitetura. Durante cada ciclo, qualquer um dos registradores pode ser escrito, mas o normal é que somente um seja escrito. Apenas um registrador pode ser copiado para o lado B da ALU. No lado A, as opções são + 1, 0, - 1, e o registrador H. Assim, podemos usar uma declaração de atribuição simples, como em Java, para indicar a operação a ser executada. Por exemplo, para copiar algo de SP para MDR, podemos dizer:
 - $MDR = SP$
- Para indicar a utilização das funções ALU, exceto passar pelo barramento B, podemos escrever, por exemplo:
 - $MDR = H + SP$

Microinstruções e Notação

Todas as operações permitidas. Qualquer uma das operações anteriores pode ser estendida somando “ $\ll 8$ ” a elas para deslocar o resultado para a esquerda por um byte. Por exemplo, uma operação comum é $H = \text{MBR} \ll 8$

$\text{DEST} = H$
$\text{DEST} = \text{SOURCE}$
$\text{DEST} = \overline{H}$
$\text{DEST} = \overline{\text{SOURCE}}$
$\text{DEST} = H + \text{SOURCE}$
$\text{DEST} = H + \text{SOURCE} + 1$
$\text{DEST} = H + 1$
$\text{DEST} = \text{SOURCE} + 1$
$\text{DEST} = \text{SOURCE} - H$
$\text{DEST} = \text{SOURCE} - 1$
$\text{DEST} = -H$
$\text{DEST} = H \text{ AND } \text{SOURCE}$
$\text{DEST} = H \text{ OR } \text{SOURCE}$
$\text{DEST} = 0$
$\text{DEST} = 1$
$\text{DEST} = -1$

Implementação da IJVM usando a Mic-1

Microprograma para a Mic-1 (1)

Rótulo	Operações	Comentários
Main1	$PC = PC + 1$; fetch; goto (MBR)	MBR contém opcode; obtenha o próximo byte; despache
nop1	goto Main1	Não faça nada
iadd1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
iadd2	$H = TOS$	H = topo da pilha
iadd3	$MDR = TOS = MDR + H$; wr; goto Main1	Some as duas palavras do topo; escreva para o topo da pilha
isub1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
isub2	$H = TOS$	H = topo da pilha
isub3	$MDR = TOS = MDR - H$; wr; goto Main1	Efetue subtração; escreva para topo da pilha
iand1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
iand2	$H = TOS$	H = topo da pilha
iand3	$MDR = TOS = MDR \text{ AND } H$; wr; goto Main1	Do AND; escreva para novas topo da pilha
ior1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
ior2	$H = TOS$	H = topo da pilha
ior3	$MDR = TOS = MDR \text{ OU } H$; wr; goto Main1	Do OR; escreva para nova topo da pilha
dup1	$MAR = SP = SP + 1$	Incremente SP e copie para MAR
dup2	$MDR = TOS$; wr; goto Main1	Escreva nova palavra da pilha
pop1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
pop2		Espere nova TOS ser lida da memória
pop3	$TOS = MDR$; goto Main1	Copie nova palavra para TOS
swap1	$MAR = SP - 1$; rd	Set MAR to $SP - 1$; leia 2a palavra da pilha
swap2	$MAR = SP$	Ajuste MAR para palavra do topo
swap3	$H = MDR$; wr	Salve TOS em H; escreva 2a palavra para topo da pilha
swap4	$MDR = TOS$	Copie TOS antigo para MDR
swap5	$MAR = SP - 1$; wr	Ajuste MAR para $SP - 1$; escreva como 2a palavra na pilha
swap6	$TOS = H$; goto Main1	Atualize TOS

Microprograma para a Mic-1 (2)

bipush1	$SP = MAR = SP + 1$	MBR = o byte para passar para a pilha
bipush2	$PC = PC + 1$; fetch	Incremente PC, busque próximo opcode
bipush3	$MDR = TOS = MBR$; wr; goto Main1	Estenda sinal da constante e passe para a pilha
iload1	$H = LV$	MBR contém índice; copie LV para H
iload2	$MAR = MBRU + H$; rd	MAR = endereço de variável local para passar para pilha
iload3	$MAR = SP = SP + 1$	SP aponta para novo topo da pilha; prepare escrita
iload4	$PC = PC + 1$; fetch; wr Inc	PC; obtenha próximo opcode; escreva topo da pilha
iload5	$TOS = MDR$; goto Main1	Atualiza TOS
istore1	$H = LV$	MBR contém índice; Copie LV para H
istore2	$MAR = MBRU + H$	MAR = endereço de variável local onde armazenar
istore3	$MDR = TOS$; wr	Copie TOS para MDR; escreva palavra
istore4	$SP = MAR = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
istore5	$PC = PC + 1$; fetch	Incremente PC; busque próximo opcode
istore6	$TOS = MDR$; goto Main1	Atualize TOS
wide1	$PC = PC + 1$; fetch;	Busque byte de operando ou próximo opcode
wide2	goto (MBR OU 0x100)	Ramificação multivias com bit alto ajustado
wide_ildoad1	$PC = PC + 1$; fetch	MBR contém 1o byte de índice; busque 2o
wide_ildoad2	$H = MBRU \ll 8$	H = 1o byte de índice deslocado 8 bits para a esquerda
wide_ildoad3	$H = MBRU \text{ OR } H$	H = índice de 16 bits de variável local
wide_ildoad4	$MAR = LV + H$; rd; goto iload3	MAR = endereço de variável local para passar
wide_istore1	$PC = PC + 1$; fetch	MBR contém 1o byte de índice; busque 2o
wide_istore2	$H = MBRU \ll 8$	H = 1o byte de índice deslocado 8 bits para a esquerda
wide_istore3	$H = MBRU \text{ OR } H$	H = índice de 16 bits de variável local
wide_istore4	$MAR = LV + H$; goto istore3	MAR = endereço de variável local no qual armazenar
ldc_w1	$PC = PC + 1$; fetch	MBR contém 1o byte de índice; busque 2o
ldc_w2	$H = MBRU \ll 8$	H = 1o byte de índice $\ll 8$
ldc_w3	$H = MBRU \text{ OR } H$	H = índice de 16 bits dentro do conjunto de constantes
ldc_w4	$MAR = H + CPP$; rd; goto iload3	MAR = endereço de constante no conjunto de constantes

Microprograma para a Mic-1 (3)

iinc1	$H = LV$	MBR contém índice; Copie LV para H
iinc2	$MAR = MBRU + H$; rd	Copie LV + índice para MAR; Leia variável
iinc3	$PC = PC + 1$; fetch	Busque constante
iinc4	$H = MDR$	Copie variável para H
iinc5	$PC = PC + 1$; fetch	Busque próximo opcode
iinc6	$MDR = MBR + H$; wr; goto Main1	Ponha soma em MDR; atualize variável
goto1	$OPC = PC - 1$	Salve endereço de opcode.
goto2	$PC = PC + 1$; fetch	MBR = 1o byte de deslocamento; busque 2o byte
goto3	$H = MBR \ll 8$	Desloque e salve primeiro byte com sinal em em H
goto4	$H = MBRU \text{ OR } H$	H= Deslocamento de desvio de 16 bits
goto5	$PC = OPC + H$; fetch	Adicione deslocamento a OPC
goto6	goto Main1	Espere para buscar o próximo opcode
iflt1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
iflt2	$OPC = TOS$	Salve TOS em OPC temporariamente
iflt3	$TOS = MDR$	Ponha novo topo da pilha em TOS
iflt4	$N = OPC$; if (N) goto T; else goto F	Desvio de bit N
ifeq1	$MAR = SP = SP - 1$; rd	Leia a palavra seguinte à do topo da pilha
ifeq2	$OPC = TOS$	Salve TOS em OPC temporariamente
ifeq3	$TOS = MDR$	Ponha novo topo da pilha em TOS
ifeq4	$Z = OPC$; if (Z) goto T; else goto F	Desvio de bit Z

Microprograma para a Mic-1 (4)

if_icmpeq1	MAR = SP = SP - 1; rd	Leia a palavra seguinte à do topo da pilha
if_icmpeq2	MAR = SP = SP - 1	Ajuste MAR para ler novo topo da pilha
if_icmpeq3	H = MDR; rd	Copie segunda palavra da pilha para H
if_icmpeq4	OPC = TOS	Salve TOS em OPC temporariamente
if_icmpeq5	TOS = MDR	Ponha novo topo da pilha em TOS
if_icmpeq6	Z = OPC - H; if (Z) goto T; else goto F	Se 2 palavras do topo da pilha iguais, goto T, else goto F
T	OPC = PC - 1; goto goto2	O mesmo que goto 1; necessário para endereço-alvo
F	PC = PC + 1	Salte primeiro byte de deslocamento
F2	PC = PC + 1; fetch	PC agora aponta para próximo opcode
F3	goto Main1	Espere por busca de opcode
invokevirtual1	PC = PC + 1; fetch	MBR = byte de índice; 1; inc. PC, obtenha 2o byte
invokevirtual2	H = MBRU << 8	Desloque e salve primeiro byte em H
invokevirtual3	H = MBRU OR H	H = deslocamento de ponteiro de método em relação a CPP
invokevirtual4	MAR = CPP + H; rd	Obtenha ponteiro para método da área CPP
invokevirtual5	OPC = PC + 1	Salve Return PC em OPC temporariamente
invokevirtual6	PC = MDR; fetch	PC aponta para novo método; obtenha contagem de parâmetros
invokevirtual7	PC = PC + 1; fetch	Busque 2o byte da contagem de parâmetro
invokevirtual8	H = MBRU << 8	Desloque e salve primeiro byte em H
invokevirtual9	H = MBRU OR H	H = número de parâmetros
invokevirtual10	PC = PC + 1; fetch	Busque 1o byte de # locais
invokevirtual11	TOS = SP - H	TOS = endereço de OBJREF - 1
invokevirtual12	TOS = MAR = TOS + 1	TOS = endereço de OBJREF (novo LV)
invokevirtual13	PC = PC + 1; fetch	Busque 2o byte de # locais
invokevirtual14	H = MBRU << 8	Desloque e salve primeiro byte em H
invokevirtual15	H = MBRU OR H	H = # locais

Microprograma para a Mic-1 (5)

invokevirtual16	$MDR = SP + H + 1; wr$	Sobrescreva OBJREF com ponteiro de enlace
invokevirtual17	$MAR = SP = MDR;$	Ajuste SP, MAR para localização para conter PC antigo
invokevirtual18	$MDR = OPC; wr$	Salve PC antigo acima das variáveis locais
invokevirtual19	$MAR = SP = SP + 1$	SP aponta para localização para conter LV antigo
invokevirtual20	$MDR = LV; wr$	Salve LV antigo acima do PC salvo
invokevirtual21	$PC = PC + 1; fetch$	Busque primeiro opcode do novo método
invokevirtual22	$LV = TOS; goto Main1$	Ajuste LV para apontar para LV Frame
ireturn1	$MAR = SP = LV; rd$	Reajuste SP, MAR para obter ponteiro de ligação
ireturn2		Espere por leitura
ireturn3	$LV = MAR = MDR; rd$	Ajuste LV para ponteiro de ligação; obtenha PC antigo
ireturn4	$MAR = LV + 1$	Ajuste MAR para ler LV antigo
ireturn5	$PC = MDR; rd; fetch$	Restaure PC; busque próximo opcode
ireturn6	$MAR = SP$	Ajuste MAR para escrever TOS
ireturn7	$LV = MDR$	Restaure LV
ireturn8	$MDR = TOS; wr; goto Main1$	Salve valor de retorno no topo de pilha original

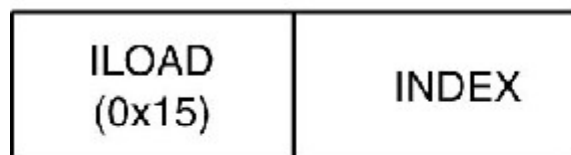
Implementação da IJVM usando a Mic-1

A instrução BIPUSH é um pouco complicada porque o opcode é seguido por um único byte, como mostra a figura. O byte deve ser interpretado como um inteiro com sinal. Esse byte, que já foi buscado para MBR em Main1, deve ser estendido em sinal para 32 bits e passado para o topo da pilha. Essa sequência deve estender em sinal o byte em MBR para 32 bits e copiá-lo para MDR. Por fim, SP é incrementado e copiado para MAR, permitindo que o operando seja escrito para o topo da pilha. No caminho, esse operando também deve ser copiado para TOS. Além disso, antes de retornar para o programa principal, note que PC deve ser incrementado de modo que o próximo opcode estará disponível em Main1.

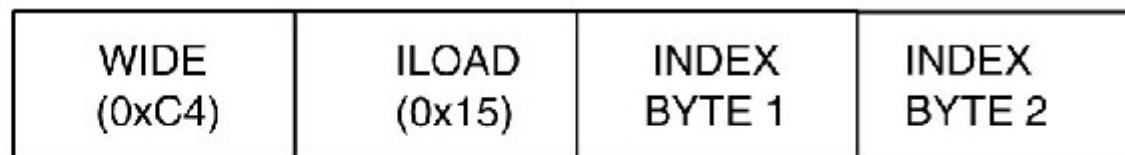
bipush1	$SP = MAR = SP + 1$
bipush2	$PC = PC + 1$; fetch
bipush3	$MDR = TOS = MBR$; wr; goto Main1



Considere a instrução ILOAD. ILOAD também tem um byte após o opcode, como mostra a figura, mas esse byte é um índice (sem sinal) para identificar a palavra no espaço de variáveis locais que será passada para a pilha. Uma vez que há somente 1 byte, apenas $2^8 = 256$ podem ser distinguidas, a saber, as primeiras 256 palavras no espaço de variáveis locais. A instrução ILOAD requer uma leitura (para obter a palavra), bem como uma escrita (para passá-la para o topo da pilha). Para determinar o endereço para leitura, entretanto, o deslocamento, contido em MBR, deve ser adicionado ao conteúdo de LV. Uma vez que ambos, MBR e LV, só podem ser acessados pelo barramento B, LV primeiro é copiado para H (em `iload1`), então MBR é adicionado. O resultado dessa adição é copiado para MAR e uma leitura é iniciada (em `iload2`).



(a)

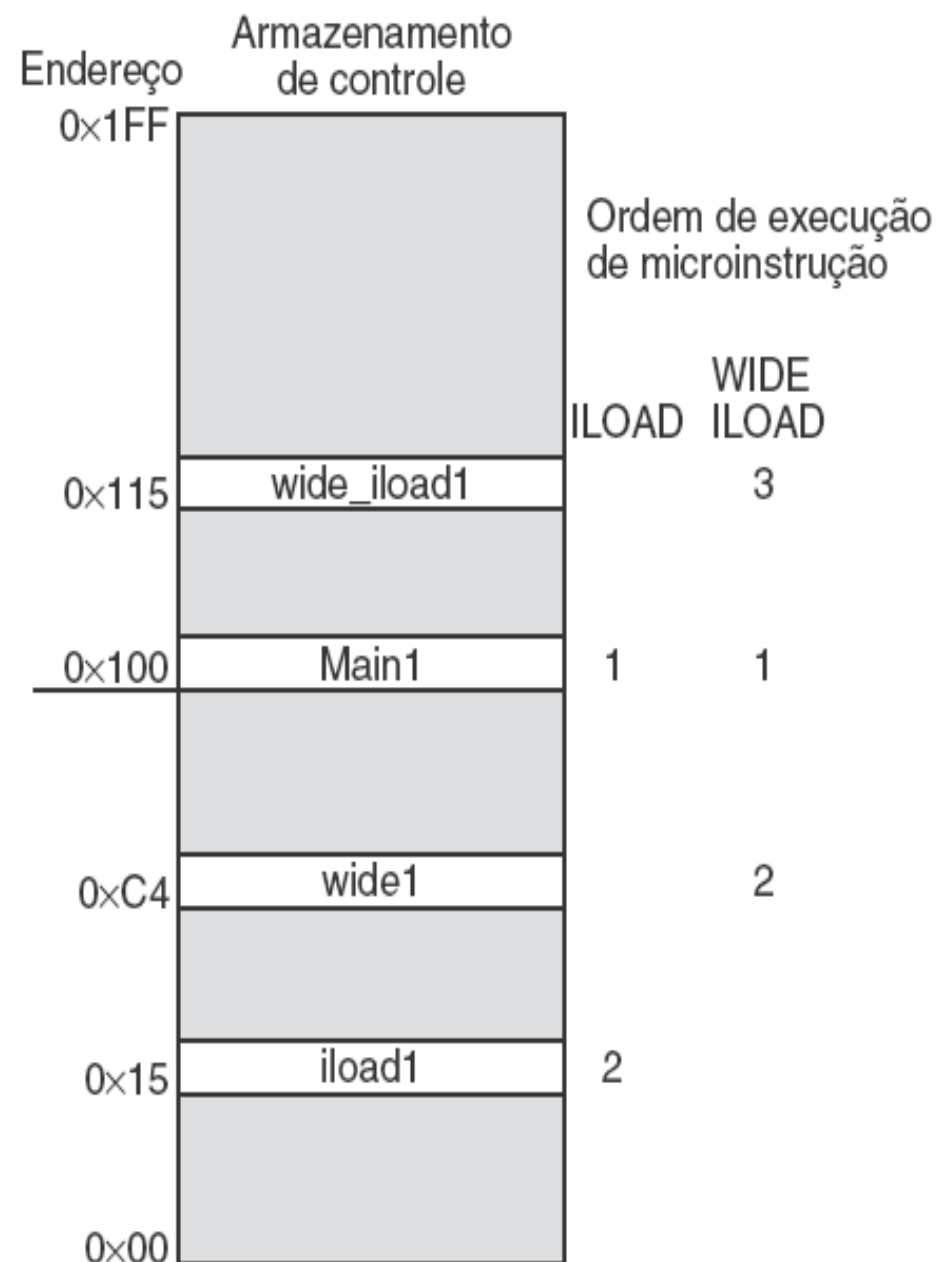


(b)

Sequência inicial de microinstruções para ILOAD e WIDE ILOAD. Os endereços são exemplos.

wide_iloa1	PC = PC + 1; fetch
wide_iloa2	H = MBRU << 8
wide_iloa3	H = MBRU OR H
wide_iloa4	MAR = LV + H; rd; goto iload3

iloa1	H = LV
iloa2	MAR = MBRU + H; rd
iloa3	MAR = SP - SP + 1
iloa4	PC = PC + 1; fetch; wr Inc
iloa5	TOS = MDR; goto Main1



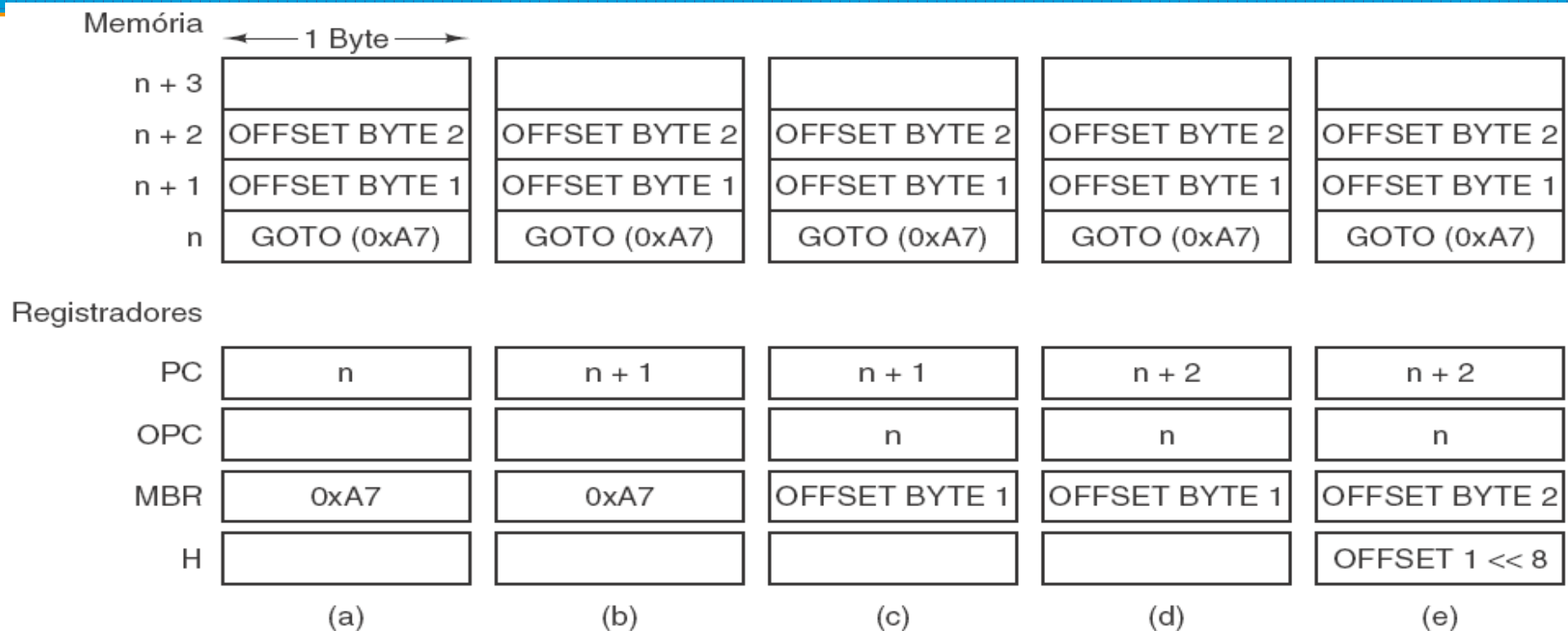
IINC (0x84)	INDEX	CONST
----------------	-------	-------

A instrução IINC é a única instrução IJVM, exceto a instrução ISTORE, que pode modificar uma variável local.

Ela o faz incluindo dois operandos, cada um de 1 byte de comprimento, como mostra a figura.

iinc1	H = LV
iinc2	MAR = MBRU + H; rd
iinc3	PC = PC + 1; fetch
iinc4	H = MDR
iinc5	PC = PC + 1; fetch
iinc6	MDR = MBR + H; wr; goto Main1

A instrução IINC usa INDEX para especificar o deslocamento em relação ao início do quadro de variáveis locais. Ela lê aquela variável, incrementando-a por CONST, um valor contido na instrução, e volta a armazená-la no mesmo local. Note que essa instrução pode incrementar por uma quantidade negativa, isto é, CONST é uma constante de 8 bits com sinal, na faixa - 128 a + 127. A JVM completa inclui uma versão ampliada da IINC onde cada operando tem 2 bytes de comprimento.



Na figura (a) vemos a situação no início de Main1. O opcode já está em MBR, mas PC ainda não foi incrementado. Na figura (b) vemos a situação no início de goto1. Nesta altura o PC já foi incrementado, mas o primeiro byte do deslocamento ainda não foi buscado para MBR. Uma microinstrução depois temos a figura (c), na qual o antigo PC, que aponta para o opcode, foi salvo em OPC e o primeiro byte do deslocamento está em MBR. Esse valor é necessário porque o deslocamento da instrução GOTOIJVM é relativa a ele e não ao valor corrente de PC. Na verdade, essa é a razão primordial por que precisamos do registrador.

Projeto no nível de microarquitetura

Velocidade contra custo

1. Reduzir o número de ciclos de relógio necessários para executar uma instrução.
2. Simplificar a organização de modo que o ciclo de relógio possa ser mais curto.
3. Sobrepor a execução de instruções.

As duas primeiras são óbvias, mas há uma surpreendente variedade de oportunidades de projeto que pode afetar drasticamente o número de ciclos de relógio, o período de relógio, ou - em grande parte das vezes - ambos.

Redução do comprimento do caminho de execução

Incorporação do laço do interpretador ao microcódigo

Rótulo	Operações	Comentários
pop1	MAR = SP = SP - 1; rd	Leia a palavra anterior à do topo na pilha
pop2		Espere novo TOS ser lido da memória
pop3	TOS = MDR; goto Main1	Copie nova palavra para TOS
Main1	PC = PC + 1; fetch; goto (MBR)	MBR contém o opcode; obtenha próximo byte; despache

Sequência original do microprograma para executar POP.

Rótulo	Operações	Comentários
pop1	MAR = SP = SP - 1; rd	Leia a palavra anterior à do topo na pilha
Main1	pop PC = PC + 1; fetch	MBR contém opcode; busque próximo byte
pop3	TOS = MDR; goto (MBR)	Copie nova palavra para TOS; despache em opcode

Sequência de microprograma melhorada para executar POP.

Incorpore o laço do interpretador ao final de cada sequência de microcódigo.

Arquitetura de três barramentos

Rótulo	Operações	Comentários
iload1	$H = LV$	MBR contém índice; Copie LV para H
iload2	$MAR = MBRU + H; rd$	MAR = endereço de variáveis locais a buscar
iload3	$MAR = SP = SP + 1$	SP aponta para novo topo da pilha; prepare escrita
iload4	$PC = PC + 1; fetch; wr$	Incremente PC; obtenha novo opcode; escreva topo da pilha
iload5	$TOS = MDR; goto Main1$	Atualize TOS
Main1	$PC = PC + 1; fetch; goto (MBR)$	MBR contém opcode; obtenha próximo byte; despache

Código Mic-1 para executar ILOAD.

Rótulo	Operações	Comentários
iload1	$MAR = MBRU + LV; rd$	MAR = endereço de variável local a buscar
iload2	$MAR = SP = SP + 1$	SP aponta para novo topo da pilha; prepare escrita
iload3	$PC = PC + 1; fetch; wr$	Incremente PC; obtenha novo opcode; escreva topo da pilha
iload4	$TOS = MDR$	Atualize TOS
iload5	$PC = PC + 1; fetch; goto (MBR)$	MBR já contém opcode; busque byte de índice

Código de três barramentos para executar ILOAD.

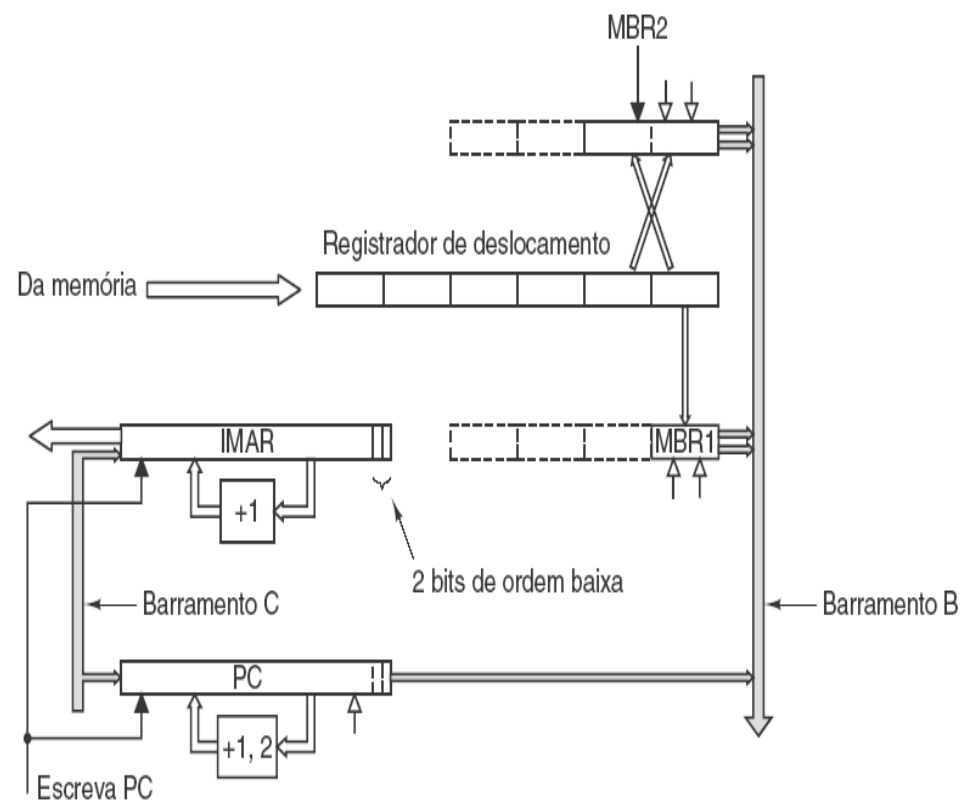
*Passe de um projeto de dois barramentos
para um projeto de três barramentos.*

Unidade de busca de instrução

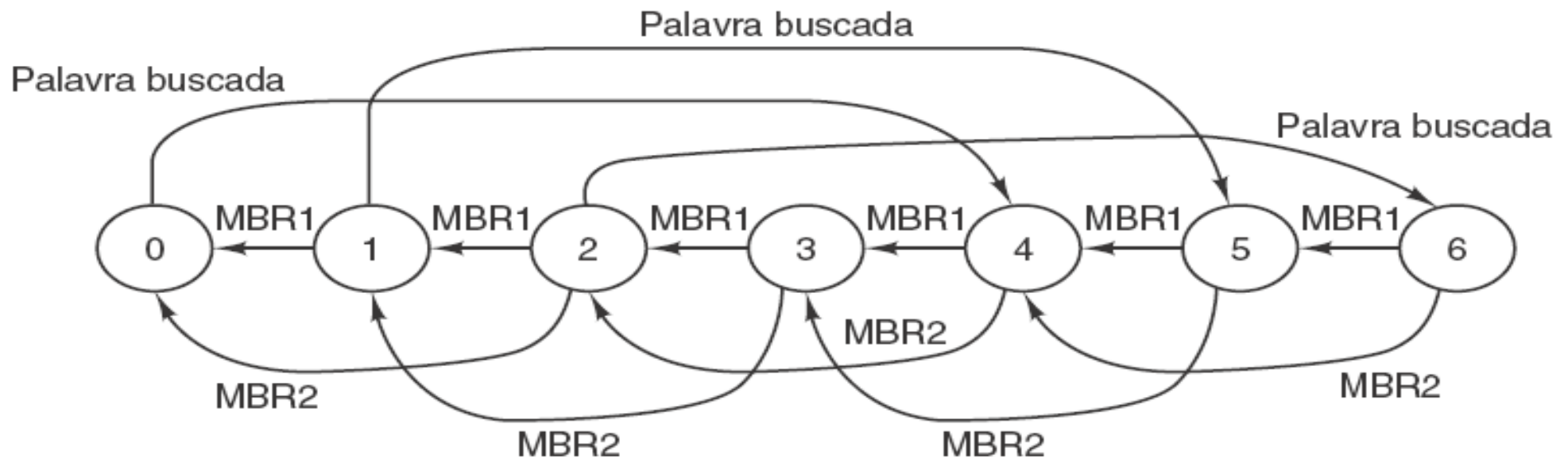
- Vale a pena usar essas duas técnicas mas, para conseguir uma melhoria drástica, precisamos de algo muito mais radical. Vamos voltar atrás um pouco e examinar as partes comuns de toda instrução: a busca e a decodificação dos campos da instrução. Observe que, para cada instrução, podem ocorrer as seguintes operações:
 1. O PC é passado pela ALU e incrementado.
 2. O PC é usado para buscar o próximo byte na sequência de instruções.
 3. Operandos são lidos da memória.
 4. Operandos são escritos para a memória.
 5. A ALU efetua um cálculo e os resultados são armazenados de volta.
- Se uma instrução tiver campos adicionais (para operandos), cada campo deve ser buscado explicitamente, 1 byte por vez, e montado antes de poder ser usado. Buscar e montar um campo ocupa a ALU por no mínimo um ciclo por byte para incrementar o PC, e então novamente para montar o índice ou deslocamento resultante. A ALU é usada em praticamente todos os ciclos para uma variedade de operações que têm a ver com buscar a instrução e montar os campos dentro da instrução, além do 'trabalho' real da instrução.

IFU (Instruction Fetch Unit)

1. A IFU pode interpretar cada opcode, determinando quantos campos adicionais devem ser buscados, e montá-los em um registrador, prontos para a utilização pela unidade de execução principal.
2. A IFU pode aproveitar a natureza sequencial das instruções e disponibilizar os próximos fragmentos de 8 e 16 bits todas as vezes, quer isso tenha ou não algum sentido. Então, a unidade de execução principal pode requisitar o que precisar.



Unidade de busca para a Mic-1.



Transições

MBR1: Ocorre quando MBR1 é lido

MBR2: Ocorre quando MBR2 é lido

Palavra buscada: ocorre quando uma palavra da memória é lida e 4 bytes são colocados no registrador de deslocamento

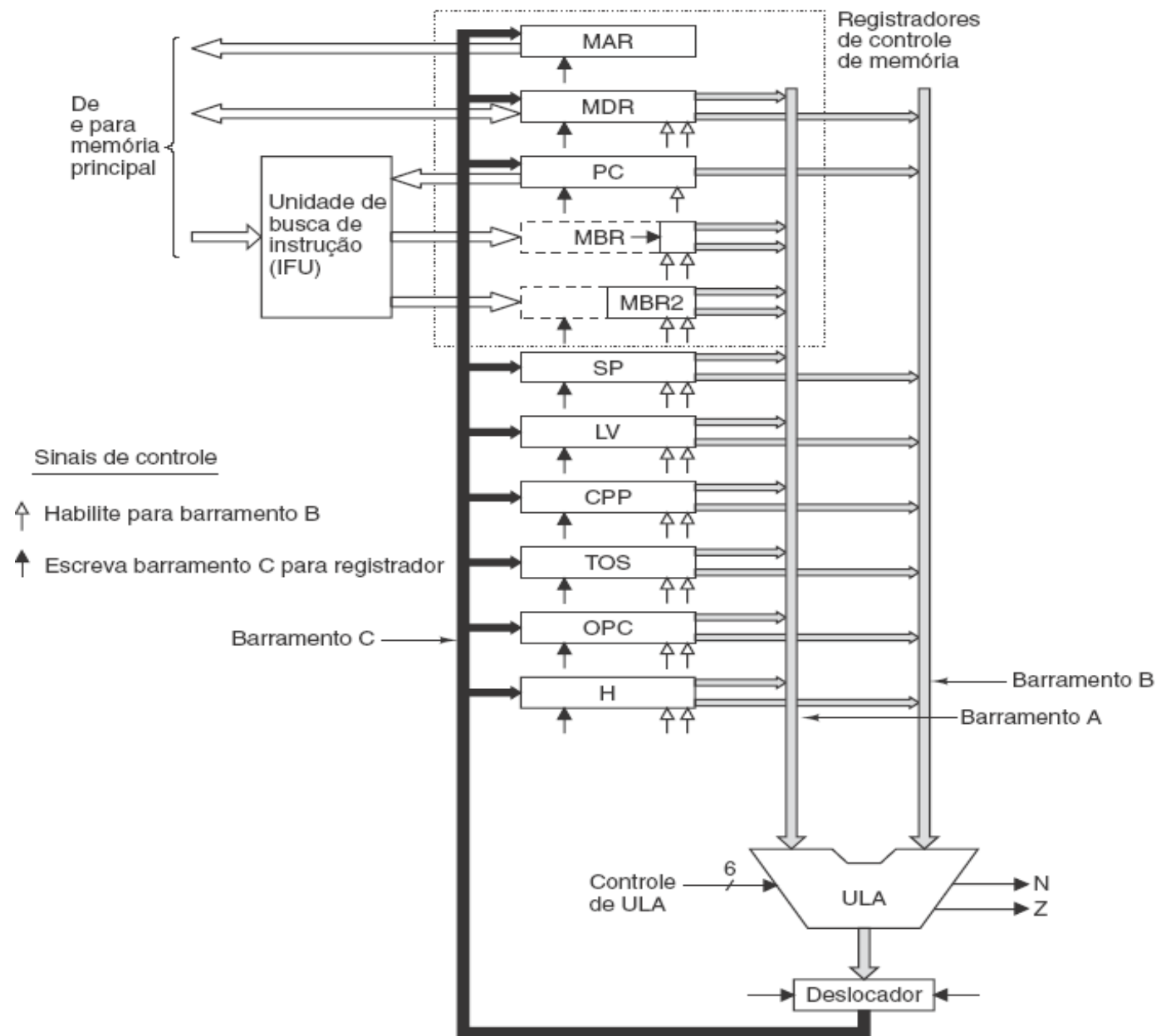
FSM (Finite State Machine)

Máquina de estado finito para implementar a IFU.

Busque instruções na memória com uma unidade funcional especializada.

O projeto com busca antecipada: a Mic-2

Caminho de dados para a Mic-2.



Rótulo	Operações	Comentários
nop1	goto (MBR)	Desvia para a próxima instrução
iadd1	MAR = SP = SP - 1; rd	Lê a palavra anterior à que está no topo da pilha
iadd2	H = TOS	H = topo da pilha
iadd3	MDR = TOS = MDR+H; wr; goto (MBR1)	Some duas palavras do topo; escreva para novo topo da pilha
isub1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha
isub2	H = TOS	H = topo da pilha
isub3	MDR = TOS = MDR-H; wr; goto (MBR1)	Subtraia TOS da palavra anterior na pilha
iand1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha
iand2	H = TOS	H = topo da pilha
iand3	MDR = TOS = MDR E H; wr; goto (MBR1)	AND palavra anterior da pilha com TOS
ior1	MAR = SP = SP - 1; rd	Leia a palavra seguinte à que está no topo da pilha
ior2	H = TOS	H = topo da pilha
ior3	MDR = TOS = MDR OU H; wr; goto (MBR1)	OR palavra anterior da pilha com TOS
dup1	MAR = SP = SP + 1	Incremente SP; copie para MAR
dup2	MDR = TOS; wr; goto (MBR1)	Escreva nova palavra da pilha
pop1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha

Microprograma para a Mic-2

Rótulo	Operações	Comentários
pop2		Espere pela leitura
pop3	TOS = MDR; goto (MBR1)	Copie nova palavra para TOS
swap1	MAR = SP - 1; rd	Leia a segunda palavra da pilha; ajuste MAR para SP
swap2	MAR = SP	Prepare para escrever nova 2a palavra
swap3	H = MDR; wr	Salve novo TOS; escreva 2a palavra para pilha
swap4	MDR = TOS	Copie antigo TOS para MDR
swap5	MAR = SP - 1; wr	Escreva antigo TOS para 2o lugar na pilha
swap6	TOS = H; goto (MBR1)	Atualize TOS
bipush1	SP = MAR = SP + 1	Ajuste MAR para escrever para novo topo da pilha
bipush2	MDR = TOS = MBR1; wr; goto (MBR1)	Atualize pilha em TOS e memória
iload1	MAR = LV + MBR1U; rd	Passe LV + index para MAR; leia operando
iload2	MAR = SP = SP + 1	Incremente SP; passe novo SP para MAR
iload3	TOS = MDR; wr; goto (MBR1)	Atualize pilha em TOS e memória
istore1	MAR = LV + MBR1U	Ajuste MAR para LV + índice
istore2	MDR = TOS; wr	Copie TOS para armazenamento
istore3	MAR = SP = SP - 1; rd	Decrementa SP; leia novo TOS
istore4		Espere por leitura
istore5	TOS = MDR; goto (MBR1)	Atualize TOS
wide1	goto (MBR1 OR 0x100)	Próximo endereço é 0x100 com operação OR efetuada com opcode
wide_ildoad1	MAR = LV + MBR2U; rd; goto iload2	Idêntica a iload1 mas usando índice de 2 bytes
wide_istore1	MAR = LV + MBR2U; goto istore2	Idêntica a istore1 mas usando índice de 2 bytes
ldc_w1	MAR = CPP + MBR2U; rd; goto iload2	O mesmo que wide_ildoad1 mas indexando a partir de CPP

Microprograma para a Mic-2

iinc1	MAR = LV + MBR1U; rd	Ajuste MAR para LV + indexe para leitura
iinc2	H = MBR1	Ajuste H para constante
iinc3	MDR = MDR + H; wr; goto (MBR1)	Incremente por constante e atualize
goto1	H = PC - 1	Copie PC para H
goto2	PC = H + MBR2	Some deslocamento e atualize PC
goto3		Tem de esperar que IFU busque novo opcode
goto4	goto (MBR1)	Despache para a próxima instrução
iflt1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha
iflt2	OPC = TOS	Salve TOS em OPC temporariamente
iflt3	TOS = MDR	Ponha novo topo da pilha em TOS
iflt4	N = OPC; if (N) goto T; else goto F	Desvie no bit N
ifeq1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha
ifeq2	OPC = TOS	Salve TOS em OPC temporariamente
ifeq3	TOS = MDR	Ponha novo topo da pilha em TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Desvie no bit Z bit
if_icmpeq1	MAR = SP = SP - 1; rd	Leia a palavra anterior à que está no topo da pilha
if_icmpeq2	MAR = SP = SP - 1	Ajuste MAR para ler novo topo da pilha
if_icmpeq3	H = MDR; rd	Copie segunda palavra da pilha para H
if_icmpeq4	OPC = TOS	Salve TOS em OPC temporariamente
if_icmpeq5	TOS = MDR	Ponha novo topo da pilha em TOS
if_icmpeq6	Z = H - OPC; if (Z) goto T; else goto F	Se 2 palavras do topo forem iguais, goto T, else goto F
T	H = PC - 1; goto goto2	O mesmo que goto1
F	H = MBR2	Toque bytes em MBR2 para descartar
F2	goto (MBR1)	

Microprograma para a Mic-2

Rótulo	Operações	Comentários
invokevirtual1	$MAR = CPP + MBR2U$; rd	Ponha endereço de ponteiro de método em MAR
invokevirtual2	$OPC = PC$	Salve Return PC em OPC
invokevirtual3	$PC = MDR$	Ajuste PC para 1o byte do código de método
invokevirtual4	$TOS = SP - MBR2U$	$TOS = \text{endereço de OBJREF} - 1$
invokevirtual5	$TOS = MAR = H = TOS + 1$	$TOS = \text{endereço de OBJREF}$
invokevirtual6	$MDR = SP + MBR2U + 1$; wr	Sobrescreva OBJREF com ponteiro de ligação
invokevirtual7	$MAR = SP = MDR$	Ajuste SP, MAR à localização para conter PC antigo
invokevirtual8	$MDR = OPC$; wr	Prepare para salvar PC antigo
invokevirtual9	$MAR = SP = SP + 1$	Incremente SP para apontar para a localização para conter LV antigo
invokevirtual10	$MDR = LV$; wr	Salve LV antigo
invokevirtual11	$LV = TOS$; goto (MBR1)	Ajuste LV para apontar para o parâmetro de ordem zero
ireturn1	$MAR = SP = LV$; rd	Reajuste SP, MAR para ler ponteiro de ligação
ireturn2		Espere por ponteiro de ligação
ireturn3	$LV = MAR = MDR$; rd	Ajuste LV, MAR para ponteiro de ligação; leia PC antigo
ireturn4	$MAR = LV + 1$	Ajuste MAR para apontar para LV antigo; leia LV antigo
ireturn5	$PC = MDR$; rd	Restaure PC
ireturn6	$MAR = SP$	
ireturn7	$LV = MDR$	Restaure LV
ireturn8	$MDR = TOS$; wr; goto (MBR1)	Salve valor de retorno no topo da pilha original

Microprograma para a Mic-2

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall